H1: Werken met Visual Studio

Labo 2 · week 1 · eerste stappen, input en output, kleuren

SEMESTER 1 · BASIS PROGRAMMEREN

Vandaag

1. **Opwarmen**: fouten zoeken
2. **De basisstructuur** van een C#-programma
3. **Output** met `WriteLine` en `Write`
4. **Input** met `ReadLine`
5. **Sneller typen** met `cw` en `cr`
6. **Kleuren** in de console

 *Vandaag laten we de computer praten. En luisteren. Meer dan sommige mensen kunnen.*

Opwarmen

Zoek de fout 🤔

Code	Fout
`Console.Writeline("Dag");`	hoofdletter: `WriteLine`
`Console.WriteLine("Dag")`	puntkomma vergeten
`Console.WriteLine(Dag);`	aanhalingstekens vergeten

😈 *Drie fouten in drie regels. Een persoonlijk record. Voor de compiler.*

De basisstructuur

Wat staat er allemaal?

```
 1  using System;
 2
 3  namespace HelloWorld
 4  {
 5      internal class Program
 6      {
 7          static void Main(string[] args)
 8          {
 9              Console.WriteLine("Hello, World!");
10          }
11      }
12  }
```

- `using System;`: we gebruiken onderdelen uit de namespace `System`, zoals `Console`.
- `namespace`: een “familienaam” die je code groepeert. Binnen een familie zijn de voornamen uniek.
- `class Program`: een klasse ordent je code. In dit semester gebruiken we ze enkel daarvoor.
- `static void Main(...)`: de **methode** waar je programma start.
- `Console.WriteLine(...);`: een **statement**, één instructie. Eindigt met een **puntkomma**.

Een namespace is een familienaam. Twee Jannen in dezelfde klas mag. Twee Jannen in hetzelfde gezin wordt verwarrend.

Sleutelwoorden

Visual Studio kleurt sommige woorden in het **blauw**: `using`, `namespace`, `class`, `static`, `void`, …

Dat zijn **sleutelwoorden** (*keywords*): gereserveerde woorden met een vaste betekenis voor C#. Je mag ze **niet** als naam gebruiken.

- `static`: de methode hoort bij de klasse zelf. In het eerste semester staat dat altijd bij onze methoden.
- `void`: de methode voert een opdracht uit, maar geeft geen antwoord terug.

😈 *Een variabele `class` noemen? C# vindt dat even grappig als jij een kind "Mama" noemen.*

Je eigen methoden

In de oefeningen schrijf je elke oefening als een **methode** in een eigen klasse:

```
1  internal class EenProgrammaSchrijvenInCSharp
2  {
3      public static void MyFirstMethod()
4      {
5          Console.WriteLine("Mijn eerste methode werkt!");
6      }
7  }
```

En in `Main` **roep je ze op**:

```
1  EenProgrammaSchrijvenInCSharp.MyFirstMethod();
```

Een methode definiëren zonder ze op te roepen: zoals een taart bakken en ze in de kast laten staan.

Output

`WriteLine` en `Write`

```
1  Console.WriteLine("Hallo");
2  Console.WriteLine("wereld");
3  Console.Write("Hallo");
4  Console.Write("wereld");
```

```
Hallo
wereld
Hallowereld
```

`WriteLine` springt na de tekst naar een **nieuwe regel**. `Write` niet: de volgende tekst komt **op dezelfde regel**.

😈 *`Write` is als een collega die nooit op Enter drukt in een chatbericht.*

Spaties tellen mee

```
1  Console.Write("Dag ");
2  Console.Write("Tim");
3  Console.Write(" hoe gaat het met je?");
```

```
Dag Tim hoe gaat het met je?
```

> ⚠ **WAARSCHUWING**
>
> Spaties tellen enkel **binnen** de aanhalingstekens. Spaties in je code, buiten de aanhalingstekens, negeert C#.

Een spatie zie je niet, maar mis je wel. Net als de wifi in de klas.

Input

`ReadLine`: de gebruiker typt

```
1  Console.WriteLine("Hoi, ik ben het!");
2  Console.WriteLine("Wie ben jij?!");
3  string name;
4  name = Console.ReadLine();
```

- `string name;` maakt een **variabele**: een plekje in het geheugen voor tekst, met de naam `name`.
- `Console.ReadLine()` wacht tot de gebruiker iets typt en op **Enter** drukt.
- Met `=` bewaar je die tekst in `name`. Dat heet een **toekenning**, en die werkt van **rechts naar links**.

 *`ReadLine` wacht geduldig tot je op Enter drukt. Een eigenschap die je bij weinig mensen vindt.*

De invoer gebruiken

```
1  Console.WriteLine("Wie ben jij?!");
2  string name = Console.ReadLine();
3  Console.WriteLine("Dag");
4  Console.WriteLine(name);
5  Console.WriteLine("hoe gaat het met je?");
```

```
Wie ben jij?!
> Tim
Dag
Tim
hoe gaat het met je?
```

Declareren en een waarde geven mag op **één regel**: `string name = Console.ReadLine();`

Je programma kent nu je naam. Dat is sneller dan sommige lectoren.

Aanhalingstekens of niet? 🤔

Stel dat de gebruiker `Tim` intypt.

Code	Output
`Console.WriteLine(name);`	`Tim`
`Console.WriteLine("name");`	`name`

Met aanhalingstekens is het letterlijke tekst. **Zonder** aanhalingstekens zoekt C# een variabele met die naam.

 *Met aanhalingstekens ben je een papegaai: je herhaalt letterlijk wat er staat.*

Aan elkaar plakken met `+`

```
1  Console.WriteLine("Dag " + name + " hoe gaat het met je?");
```

```
Dag Tim hoe gaat het met je?
```

En met meer invoer, telkens een **eigen** variabele:

```
1  Console.WriteLine("Hoe oud ben je?");
2  string age = Console.ReadLine();
3  Console.WriteLine("Waar woon je?");
4  string city = Console.ReadLine();
```



Tekst plakken met `+`: knutselen voor programmeurs. Lijm niet inbegrepen.

Sneller typen

`cw` en `cr`

- Typ `cw` en druk twee keer op **Tab**: Visual Studio maakt er `Console.WriteLine();` van.
- `cr` voor `Console.ReadLine();` zit niet standaard in Visual Studio. Je voegt het zelf toe:
  1. Download het snippet-bestand van de cursuspagina (of kopieer de code in een bestand `cr.snippet`).
  2. **Tools** → **Code Snippets Manager** → kies **CSharp** → **Import**.

 `cw` + *Tab Tab: twaalf toetsaanslagen bespaard. Tijd genoeg voor een extra koffie per dag.*

Kleuren

Tekst in kleur

```
1  Console.WriteLine("Tekst in de standaardkleur");
2  Console.BackgroundColor = ConsoleColor.Blue;
3  Console.ForegroundColor = ConsoleColor.Green;
4  Console.WriteLine("Groen op een blauwe achtergrond");
5  Console.ForegroundColor = ConsoleColor.Red;
6  Console.WriteLine("Rood op een blauwe achtergrond");
```

- `ForegroundColor`: de kleur van de **letters**
- `BackgroundColor`: de kleur van de **achtergrond**
- Een kleur blijft gelden voor **alle tekst die erna** komt, tot je ze verandert.



Groen op blauw: ideaal voor wie zijn gebruikers wil laten huilen.

Kleuren resetten

```
1  Console.ForegroundColor = ConsoleColor.Red;
2  Console.WriteLine("Fout! Contacteer de helpdesk.");
3  Console.ResetColor();
4  Console.WriteLine("Het programma sluit nu af.");
```

`Console.ResetColor()` zet de **oorspronkelijke** kleuren van de gebruiker terug.

> **TIP**
>
> Gebruik kleur met een **doel**, bijvoorbeeld foutmeldingen in het rood. Zet de kleuren daarna altijd terug.

`ResetColor`: *de knop "alles terug zoals het was". Bestond die maar voor maandagochtend.*

Welke kleur? 🤔

```
1  Console.ForegroundColor = ConsoleColor.Yellow;
2  Console.WriteLine("Regel 1");
3  Console.WriteLine("Regel 2");
4  Console.ForegroundColor = ConsoleColor.Cyan;
5  Console.WriteLine("Regel 3");
6  Console.ResetColor();
7  Console.WriteLine("Regel 4");
```

- Regel 1 → **geel**
- Regel 2 → **geel**, want de kleur blijft gelden
- Regel 3 → **cyaan**
- Regel 4 → de **standaardkleur**



Geel, geel, cyaan, standaard. Klinkt als een verkeerslicht dat het opgegeven heeft.

Samengevat

Wat	Hoe
tekst tonen, nieuwe regel	`Console.WriteLine("...");`
tekst tonen, zelfde regel	`Console.Write("...");`
invoer lezen	`string name = Console.ReadLine();`
tekst plakken	`"Dag " + name`
kleur van de letters	`Console.ForegroundColor = ConsoleColor.Red;`
kleuren terugzetten	`Console.ResetColor();`



Je programma kan nu praten, luisteren én in kleur roepen. Eén les en al bijna een tiener.

Aan de slag 💻

Maak in de klasse `EenProgrammaSchrijvenInCSharp`:

1. **MyFirstProgram**: vraag voor- en achternaam en toon ze in twee volgordes
2. **Rubbish**: stel vijf vragen en toon de antwoorden bewust door elkaar
3. **ColoredRubbish**: hetzelfde, maar elke regel in een andere kleur
4. **AddressCard**: stel een naamkaartje samen

> 😈 *Rubbish: het enige programma waarvoor je punten krijgt als het onzin toont.*